

ADVANCED OPERATING SYSTEMS

Lab 4 — Kernel allocation and userspace access

Vittorio Zaccaria

Politecnico di Milano

Overview

What you will build

One kernel module, written in two stages, plus one small userspace program.

- **Part A — kernel memory.** Allocate memory with `kmalloc`, `vmalloc` and a slab cache, and print where it lives in physical memory.
- **Part B — the userspace boundary.** Expose `/proc/lab4` and move bytes between a process and the kernel with `copy_from_user` / `copy_to_user`, including a copy that runs into an inaccessible page.

Each stage ends with a build and a **checkpoint** in QEMU. Answer the questions on each checkpoint slide before moving on.

The two parts answer two questions: does a virtual address tell you where data sits physically (A)? Can the kernel trust a pointer that comes from a process (B)?

Set up the module

Create a working module next to the others:

```
# Host
mkdir -p modules/lab-4-work
printf '%s\n' 'include $(dir $(lastword $(MAKEFILE_LIST)))../common.mk' \
  > modules/lab-4-work/Makefile
```

Write `modules/lab-4-work/module.c`, starting from the Lab 1 hello module (`MODULE_LICENSE("GPL")`, `module_init`, `module_exit`). `make dev-build` finds the directory and installs `/modules/lab-4-work.ko` in the guest.

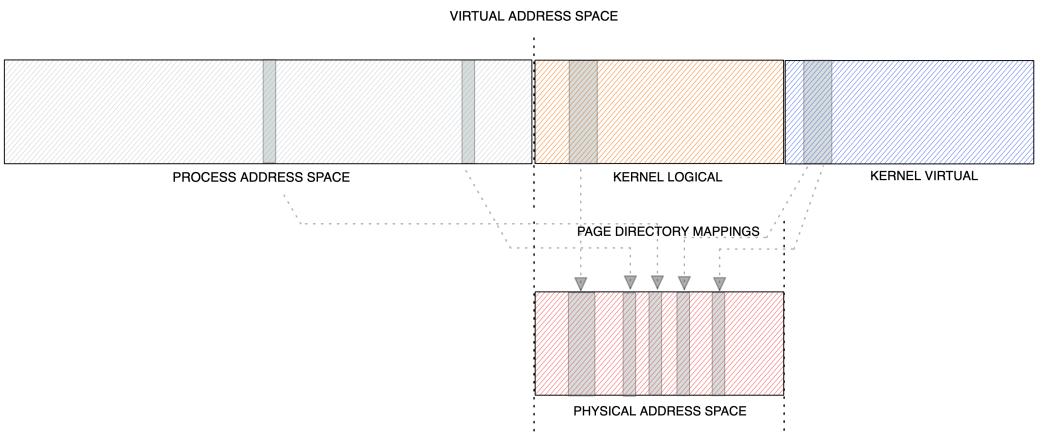
Use either frontend from Lab 1; one vCPU is enough:

```
T=amd64 E=full make dev-up      # once
T=amd64 E=full make dev-build   # after every change
T=amd64 E=full make dev-run
```

Exit QEMU with `Ctrl-]`, then `x`.

Part A — Kernel pages and objects

A1 · Task: four pages, two allocators



In the init function, allocate `4 * PAGE_SIZE` bytes twice: once with `kmalloc(..., GFP_KERNEL)` and once with `vmalloc`. For each of the four pages of each buffer, print its virtual address and its **PFN** (page frame number: physical address `>> PAGE_SHIFT`):

```
kmalloc[i] va=<address> pfn=<pfn>
vmalloc[i] va=<address> pfn=<pfn>
```

A1 · Useful APIs

NEED	API
physically contiguous buffer	`kmalloc(size, GFP_KERNEL)`, `kfree`
virtually contiguous buffer	`vmalloc(size)`, `vfree` (`<linux/vmalloc.h>`)
PFN of a `kmalloc` address	`virt_to_phys(p) >> PAGE_SHIFT` (`<asm/io.h>`)
PFN of a `vmalloc` address	`page_to_pfn(vmalloc_to_page(p))`
print a kernel pointer unchanged	`%px` (plain `%p` prints a hash)

`virt_to_phys` works only for the kernel's direct mapping, where `kmalloc` memory lives. A `vmalloc` address must go through its page table: `vmalloc_to_page` finds the `struct page` behind one virtual page.

Return `-ENOMEM` if an allocation fails. Free both buffers before init returns. **Before running**, predict the four `kmalloc` PFNs and the four `vmalloc` PFNs.

A1 · Checkpoint

```
# Guest
insmod /modules/lab-4-work.ko
dmesg | grep -E 'kmalloc\[|vmalloc\['
rmmod module
```

Example from one run (addresses and PFNs vary):

```
kmalloc[0] va=ffff888006110000 pfn=6110
kmalloc[1] va=ffff888006111000 pfn=6111
vmalloc[0] va=ffffc90000228000 pfn=6103
vmalloc[1] va=ffffc90000229000 pfn=6102
```

Questions.

1. By how much do consecutive virtual addresses advance in each buffer?
2. Which PFN sequence is consecutive? Is that guaranteed, or just what happened?
3. Why does `kmalloc` become a poor choice for very large buffers?

A2 · Task: a slab cache with a constructor

Continue in the same init function, after A1:

1. Define `struct sample { u64 value; };` and a constructor `void construct(void *object)` that sets `value` to 0 and increments a global counter `constructed`.
2. Create a cache: `kmem_cache_create("lab4_objects", sizeof(struct sample), 0, 0, construct)`.
3. Allocate **one** object with `kmem_cache_alloc(cache, GFP_KERNEL)`.
4. Print `cache allocated=1 constructed=<counter>`.
5. Free the object with `kmem_cache_free` and destroy the cache with `kmem_cache_destroy` before init returns.

Handle a `NULL` cache or object with `-ENOMEM`, destroying what was already created.

A2 · Checkpoint

```
# Guest
insmod /modules/lab-4-work.ko
dmesg | grep 'cache allocated'
rmmod module
```

Example from one run:

```
cache allocated=1 constructed=256
```

Questions.

1. One object was allocated. Why was the constructor called many more times?
2. What does the number depend on?
3. A constructor runs when a slot is first prepared, not at every allocation. What must an object look like when it is freed back to the cache?

Part B — Crossing the userspace boundary

B1 · Task: a procfs endpoint

Extend the same module (`<linux/proc_fs.h>` , `<linux/uaccess.h>`). procfs is only a door: the shortest way to make a `read(2)` or `write(2)` reach your code.

- At the end of init, create the file with `proc_create("lab4", 0600, NULL, &ops)` (root only, no parent directory). Return `-ENOMEM` if it fails.
- In exit, remove it with `proc_remove` .
- Connect the system calls to two callbacks:

```
static const struct proc_ops ops = {
    .proc_read = send,     /* read(2):  the kernel sends to the process    */
    .proc_write = receive, /* write(2): the kernel receives from the process */
};
```

Both callbacks get a `__user` pointer: an address in the calling process. **Never dereference it.** Use `copy_from_user` / `copy_to_user` , which return the number of bytes **not** copied.

B1 · Specification of `receive` and `send`

```
ssize_t receive(struct file *file, const char __user *user,
                size_t count, loff_t *offset);
ssize_t send(struct file *file, char __user *user,
             size_t count, loff_t *offset);
```

`receive` (write):

- more than 16 bytes: return `-E2BIG`; zero bytes: return 0;
- copy into a 16-byte buffer on the stack and log `from_user requested=<count> uncopied=<n>`;
- if any byte was not copied, return `-EFAULT`; otherwise log `received=<text>` and return `count`. The text has no final `\0`: print it with `%.*s`.

`send` (read). The file content is the fixed string `kernel!\n`:

- if `*offset` is at or past its end, return 0 (end of file);
- copy at most the bytes left after `*offset` and log `to_user requested=<n> uncopied=<m>`;
- on a partial copy return `-EFAULT` and leave `*offset` unchanged; otherwise advance `*offset` and return the number of bytes copied.

B1 · Checkpoint

```
# Guest
insmod /modules/lab-4-work.ko
printf ciao > /proc/lab4
cat /proc/lab4
dmesg | grep -E 'from_user|to_user|received='
```

Expected: `cat` prints `kernel!`, and the log shows

```
from_user requested=4 uncopied=0
received=ciao
to_user requested=8 uncopied=0
```

Questions.

1. Why does `cat` stop after one line? What would happen if `send` ignored `*offset`?
2. Why must the length check in `receive` come before the copy?

Exit QEMU before B2: the next step rebuilds the initramfs.

B2 · Task: a copy that runs into an inaccessible page

Write `modules/lab-4-work/user-demo.c`, a userspace program that:

1. maps two anonymous read-write pages with `mmap`;
2. makes the **second** page inaccessible with `mprotect(..., PROT_NONE)`;
3. writes `ABCD` in the last 4 bytes of the first page;
4. opens `/proc/lab4` and, with `p` pointing 4 bytes before the page boundary, calls `write(fd, p, 8)` and then `read(fd, p, 8)`;
5. prints `write=<result> errno=<errno>; read=<result> errno=<errno>`;
6. exits 0 only if both calls returned −1 with `errno == EFAULT`.

```
        first page (read-write) | second page (PROT_NONE)
  . . . . . . . . . . . . A B C D|. . . . . . . . . . .
                        ^ p    ^ page boundary
                        <-4 ok><-- 4 not accessible -->
```

B2 · Build and run

The program runs in the guest, so it is compiled statically into the initramfs:

```
# Host, after make dev-build
docker exec lkp-dev-amd64 gcc -static -O2 -Wall -Wextra -Werror \
  /repo/modules/lab-4-work/user-demo.c \
  -o /staging/initramfs/fs/lab4-user-demo
docker exec -e __BUILD_ARCH=amd64 lkp-dev-amd64 /sources/mini-bootstrap.sh
T=amd64 E=full make dev-run
```

Browser frontend: run the same `gcc` command with `sudo` in the code-server terminal, then `sudo env __BUILD_ARCH=amd64 /sources/mini-bootstrap.sh`, then the QEMU task.

```
# Guest
insmod /modules/lab-4-work.ko
/lab4-user-demo; echo status=$?
dmesg | grep -E 'from_user|to_user'
rmmod module
```

B2 · Checkpoint

Expected:

```
from_user requested=8 uncopied=4
to_user requested=8 uncopied=4
write=-1 errno=14; read=-1 errno=14
status=0
```

`errno=14` is `EFAULT` on Linux.

Questions.

1. Why exactly 4 uncopied bytes? What would you expect with `p` two bytes before the boundary?
2. The kernel took a page fault inside `copy_from_user`. Why did it not oops, as the null-pointer module did in Lab 1?
3. After the failed `read`, can the first 4 bytes at `p` have changed?

Wrap-up

Done when

- `insmod` prints four `kmalloc` lines, four `vmalloc` lines and the cache line;
- `/proc/lab4` accepts a short write, returns `kernel!` once, and disappears after `rmmod module`;
- `/lab4-user-demo` exits with status 0;
- `dmesg | grep -E 'BUG:|WARNING:|Oops:'` prints nothing.